

Programming in Java Project Report

Name - Ayush Raj

Registration No. – 25BAI11407

Branch - Cse AI&ML

Course – Programming in Java

Year – 2nd year

Date – 15 September 2026

SR . No.	Table of Contents	Page
1	Introduction	3
2	Problem Statement	4
3	Objectives	5
4	Functional Requirements	6
5	Non - Functional Requirements	7
6	System Architecture	8
7	Design Diagrams	9
8	Design Decisions & Rationale	12
9	Implementation Details	13
10	Results	17
11	Testing Approach	20
12	Challenges Faced	22
13	Learnings & Key Takeaways	23
14	Future Enhancements	24
15	References	25

Introduction

Hostel management involves handling several day-to-day activities such as maintaining student records, managing rooms, tracking hostel fees, and addressing student complaints. When these activities are handled manually or through disconnected processes, it can become difficult to maintain accurate records and respond to issues efficiently.

Hostel Sync is a Java-based, command-line hostel management application developed to provide a centralized and organized way of handling common hostel-related activities. The system provides role-based access for students and faculty through a structured console-based workflow., allowing users to access relevant information and perform hostel operations through a structured console-based workflow.

The application includes user login and role selection, hostel and room information, fee tracking, complaint submission, complaint history, and complaint status

management. One of the key features of Hostel Sync is its complaint classification functionality, which analyzes the complaint description and automatically assigns it to an appropriate category such as Plumbing, Electrical, Cleanliness, Internet, Furniture, or Other.

For faculty users, the system provides a dashboard that summarizes complaint information and allows complaints to be viewed and their statuses to be updated. This helps provide a clearer overview of hostel maintenance issues and their current status.

The project is implemented using **Java and Maven** with a modular structure consisting of models, services, repositories, and application components.

Overall, Hostel Sync aims to demonstrate how software can be used to organize hostel operations, simplify complaint management, improve accessibility of information, and provide a structured workflow for both students and administrators.

Problem Statement

Hostel residents often need to report issues related to plumbing, electrical systems, cleanliness, internet connectivity, furniture, and other hostel facilities. Managing these complaints manually can make it difficult to organize requests, identify their categories, track their progress, and maintain a clear record of previously submitted complaints.

In addition to complaint management, students may need access to basic hostel information, room details, and fee status, while faculty users require an overview of complaints and their current statuses. Without a centralized system, these activities can become fragmented and time-consuming.

The problem addressed by Hostel Sync is the lack of a simple and organized system for

managing these common hostel operations through a single application. The project aims to provide a structured platform where students can access hostel-related information and submit and track complaints, while administrators can monitor complaints, analyze their distribution, and update their statuses.

Therefore, Hostel Sync is designed to bring these activities together into one centralized application, making hostel-related information and complaint management more organized, accessible, and manageable.

Objectives

The main objectives of Hostel Sync are:

1. To centralize hostel-related information such as student details, room information, and fee records in a structured application.
2. To provide an organized complaint management system that allows students to submit complaints and view their complaint history and current status.
3. To automatically classify complaints into relevant categories such as Plumbing, Electrical, Cleanliness, Internet, Furniture, and Other based on the complaint description.
4. To provide administrators with an overview of hostel complaints through a dashboard containing complaint and category statistics.

5. To enable complaint status management, allowing administrators to track and update the progress of submitted complaints.
6. To provide a simple login and role-selection mechanism for accessing the application..
7. To develop a modular and maintainable Java application that demonstrates the practical application of programming, object-oriented design, data management, and software development concepts.
8. To provide a simple command-line workflow that can be easily executed and used without requiring a graphical interface.

Functional Requirements

The functional requirements of Hostel Sync describe the main operations and services provided by the application.

1. User Authentication

- . The system shall provide login functionality for authorized students and administrators.
- . Passwords shall be handled using password hashing rather than storing them as plain text.
- . The system shall provide appropriate feedback when invalid credentials are entered.

2. Student Management

- . The system shall maintain student information.

- . The system shall allow students to access relevant hostel information after authentication.
- . The system shall provide access to room and fee-related information.

3. Room and Hostel Information

- . The system shall maintain hostel and room information.
- . The system shall allow users to view available hostel-related details through the application.
- . The system shall associate students with their respective room information.

4. Fee Management

- . The system shall maintain hostel fee records.

- The system shall allow the fee status of a student to be viewed.
- The system shall distinguish between paid and unpaid fee records.

5. Complaint Management

- The system shall allow students to submit hostel-related complaints.
- The system shall record the submitted complaint details.
- The system shall allow students to view their complaint history.
- The system shall display the current status of submitted complaints.
- The system shall allow administrators to view all submitted complaints.
- The system shall allow administrators to update complaint statuses.

6. Complaint Classification

- . The system shall analyze the description provided with a complaint.
- . The system shall automatically assign the complaint to a relevant category.
- . The supported categories include Plumbing, Electrical, Cleanliness, Internet, Furniture, and Other.
- . The classification shall help organize complaints for easier management.

7. Administrator Dashboard

- . The system shall provide administrators with a dashboard for monitoring complaints.
- . The dashboard shall display the total number of complaints.

- The dashboard shall provide category-wise complaint information.
- The dashboard shall provide status-wise complaint information.
- Administrators shall be able to use the dashboard to obtain an overview of the current complaint situation.

8. Data Management

- The system shall organize application data using separate models, services, and repository components.
- The system shall provide appropriate validation and error handling for user operations.
- The application shall maintain a structured workflow for accessing and modifying hostel-related information

Non-Functional Requirements

1. Security

The system shall protect user access through authentication and password hashing. Only authenticated users shall be able to access the functionalities available to their respective roles.

2. Usability

The application shall provide a simple and structured command-line interface. Menus and options shall be presented clearly so that students and administrators can navigate the available functions easily.

3. Reliability

The system shall handle invalid inputs and unsuccessful operations appropriately and provide suitable feedback to the user. Application operations shall be managed consistently throughout the workflow.

4. Maintainability

The application shall follow a modular structure by separating models, services, repositories, and application components. This structure makes the system easier to understand, maintain, and extend.

5. Error Handling

The system shall validate user inputs and handle invalid credentials, incorrect menu selections, and other invalid operations without terminating the application unexpectedly.

6. Resource Efficiency

The application shall use a lightweight command-line interface and modular processing so that hostel operations can be performed without requiring extensive system resources.

System Architecture

Hostel Sync follows a modular, layered architecture that separates the application into different components based on their responsibilities. This structure helps organize the application logic and makes the system easier to maintain and extend.

The main components of the Hostel Sync architecture are:

1. User Interface Layer

The application provides a command-line interface through which students and administrators interact with the system. Users can log in, select available operations, enter required information, and view the results generated by the application.

2. Application Layer

The application layer controls the main workflow of Hostel Sync. It manages the application menu, user interaction,

authentication flow, and navigation between different operations.

3. Service Layer

The service layer contains the main business logic of the application. It handles operations such as complaint submission, complaint classification, complaint status management, fee-related operations, and dashboard information.

4. Model Layer

The model layer represents the main entities used by the system. These entities store and organize information related to students, rooms, complaints, fees, and other hostel-related data.

5. Repository Layer

The repository layer manages the storage and retrieval of application data. It provides a

separate layer for accessing and managing the information used by the application.

6. Complaint Classification Component

Hostel Sync includes a complaint classification component that analyzes complaint descriptions and assigns them to appropriate categories such as Plumbing, Electrical, Cleanliness, Internet, Furniture, or Other.

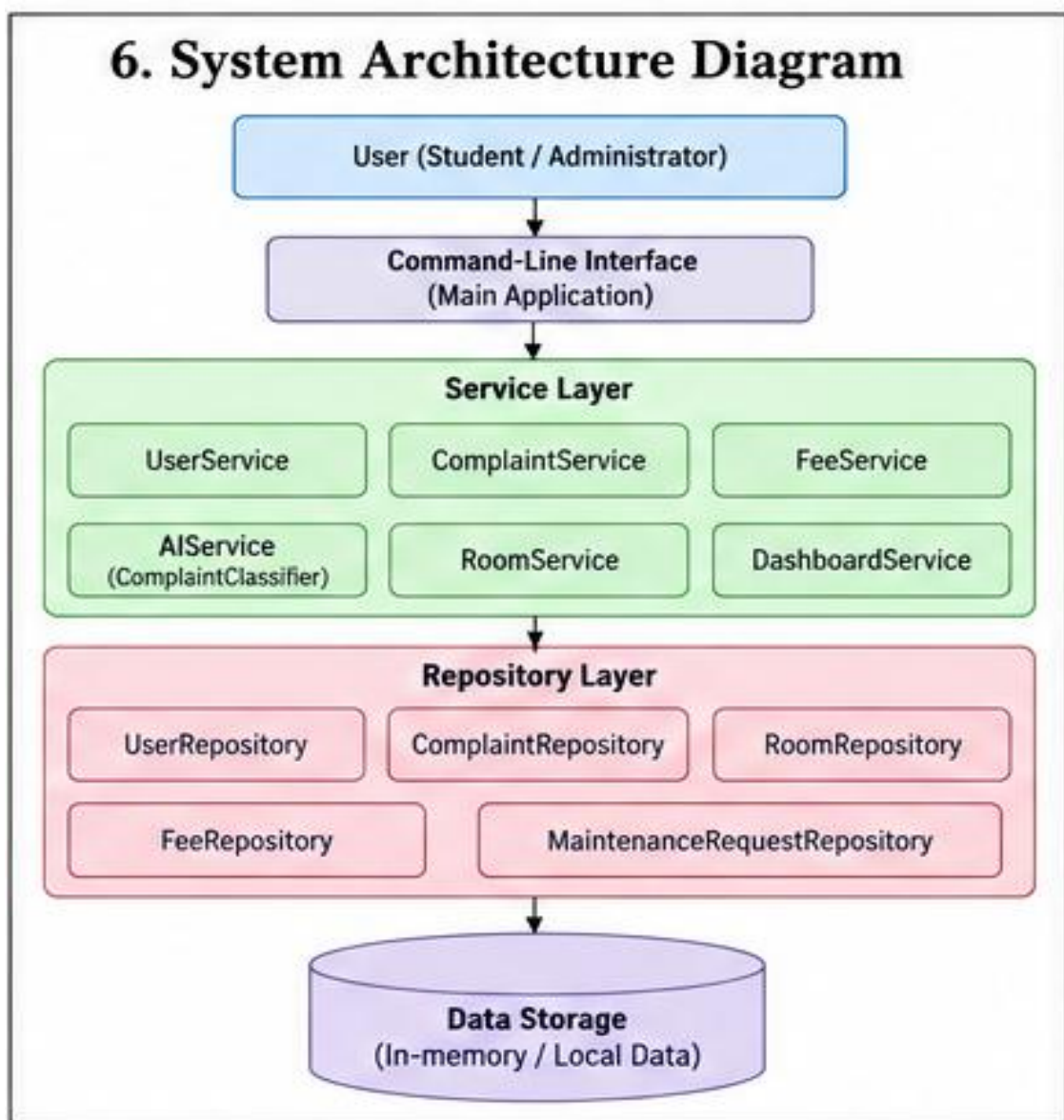
7. Authentication Component

The authentication component manages user login and verifies credentials before allowing access to the application. Password hashing is used to handle passwords securely.

Overall Architecture Flow

Student / Administrator → Command-Line Interface → Application Layer → Service Layer → Repository Layer → Data

The complaint classification and authentication components support the relevant operations within the application workflow.



Design Diagrams

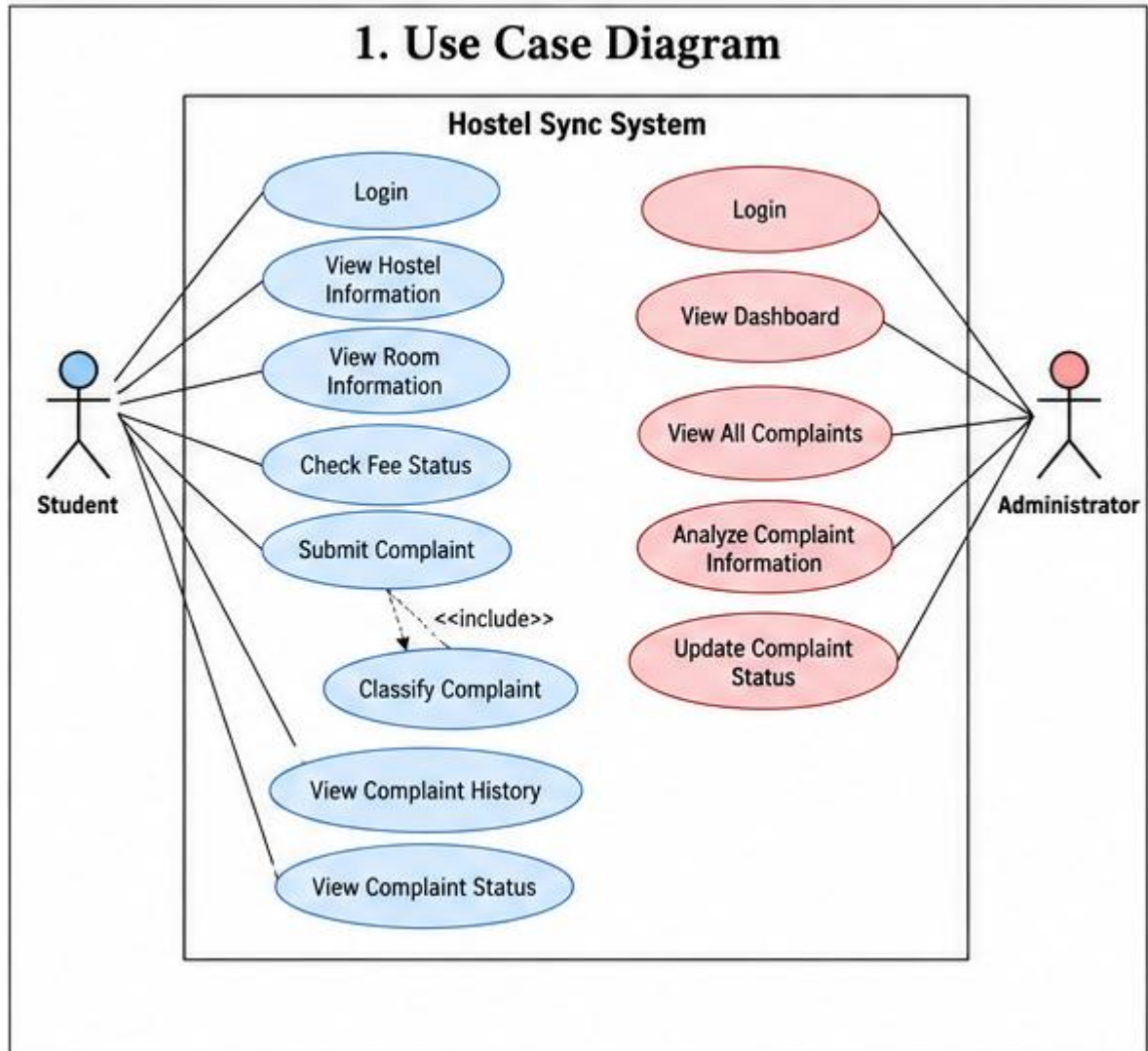
This section presents the design diagrams used to describe the structure, workflow, interactions, and components of the Hostel Sync application.

1. Use Case Diagram

The Use Case Diagram represents the main interactions between the two primary users of Hostel Sync: **Student** and **Administrator**.

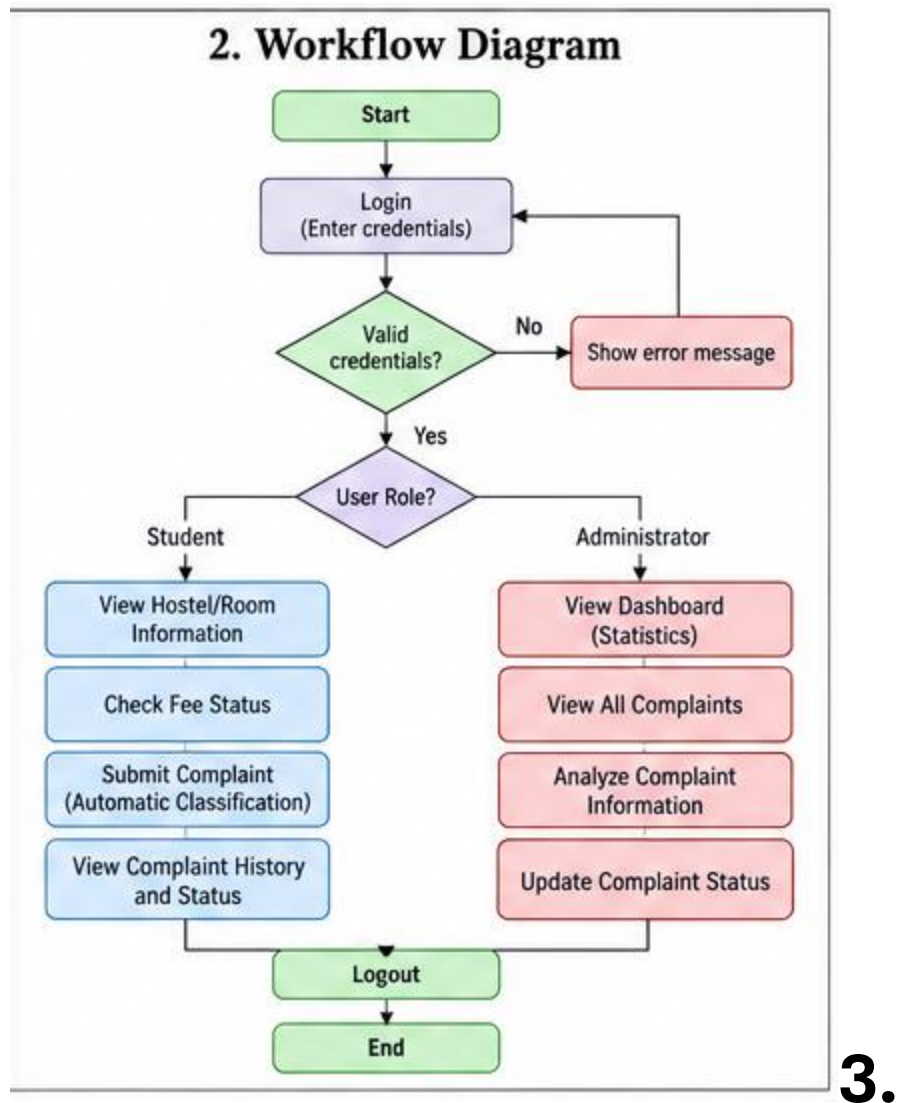
The Student can authenticate, view hostel and room information, check fee status, submit complaints, view complaint history, and check complaint status.

The Administrator can authenticate, access the dashboard, view complaints, analyze complaint information, and update complaint statuses.



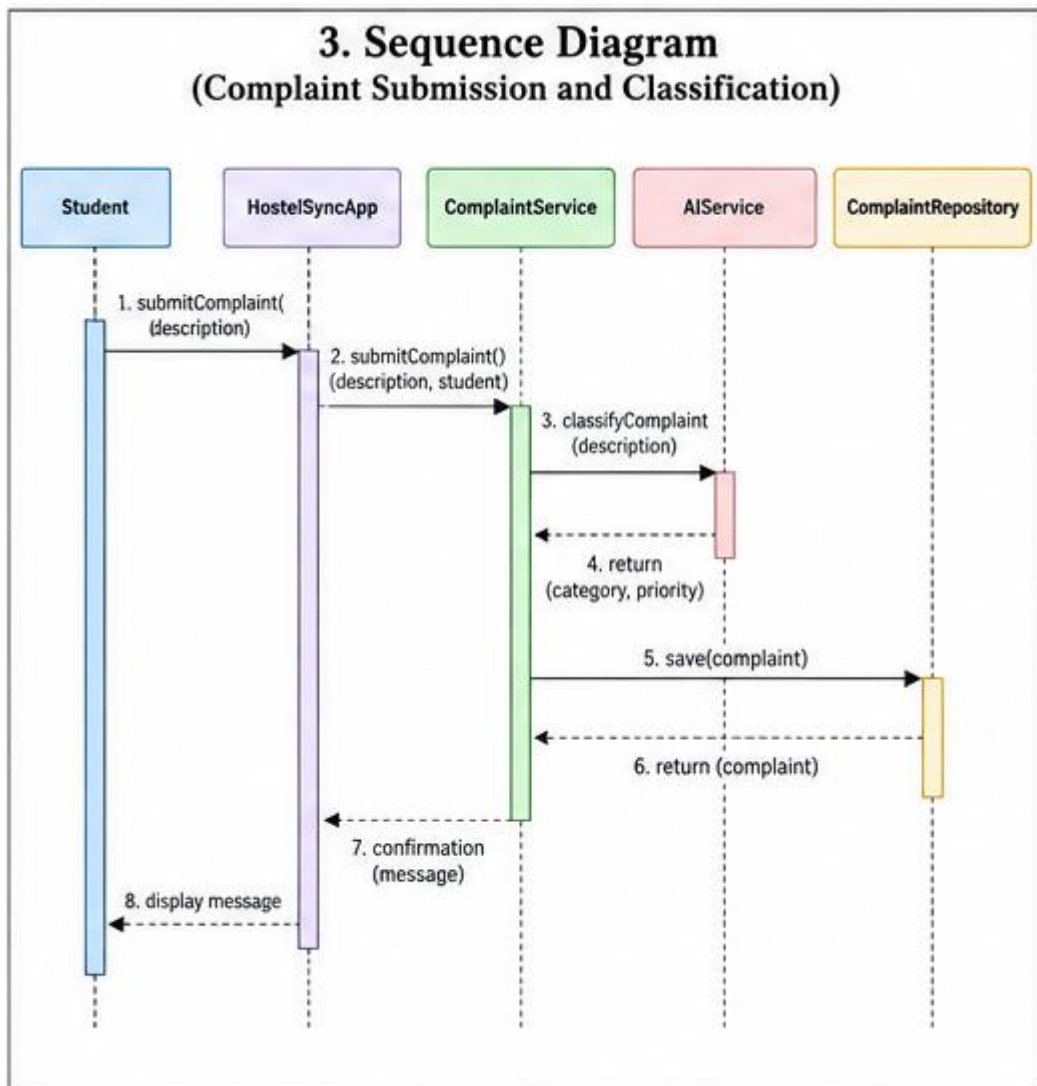
2. Workflow Diagram

The Workflow Diagram represents the overall sequence of operations performed in Hostel Sync, beginning with user authentication and continuing through the available student or administrator functions.



3. Sequence Diagram

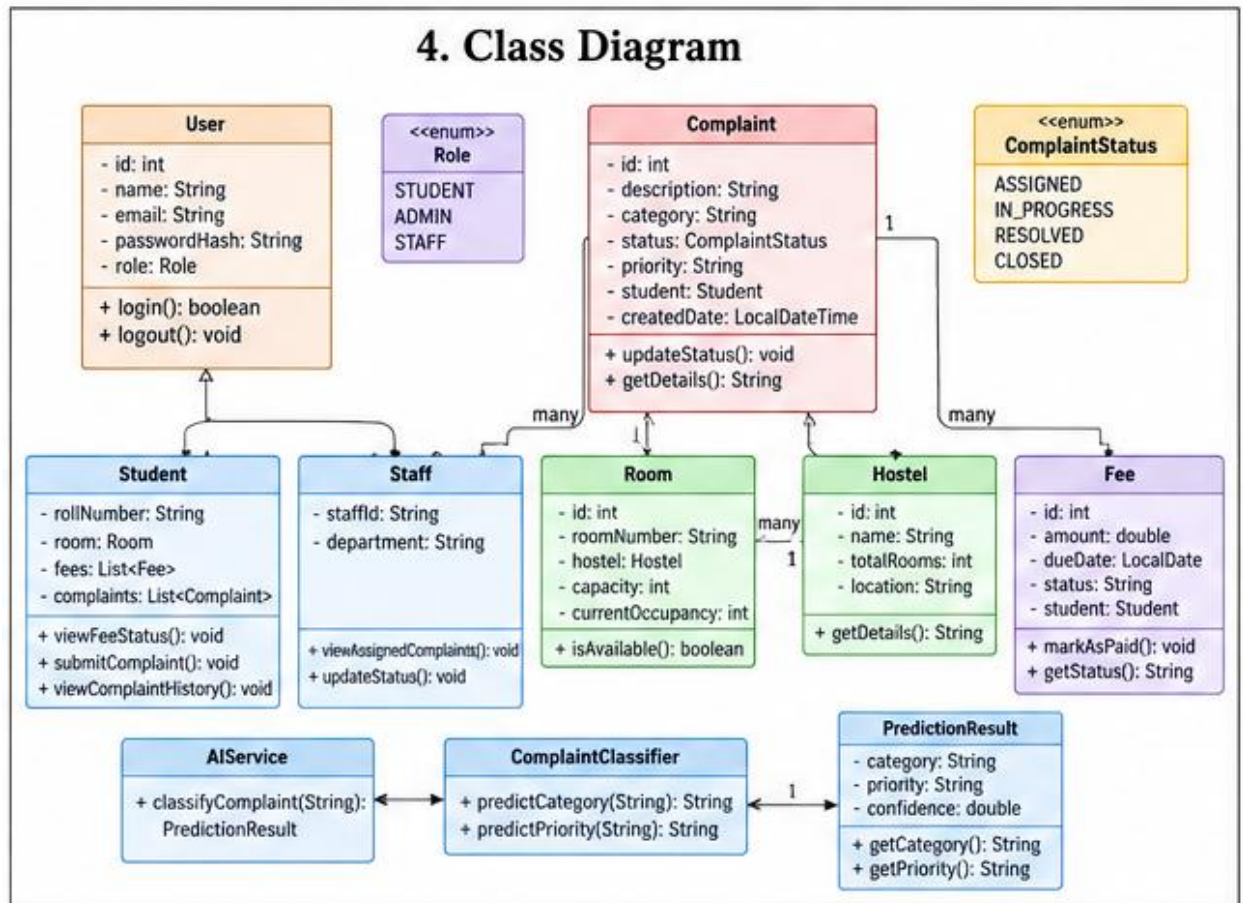
The Sequence Diagram represents the interaction between the user, application components, services, and data repositories during important operations such as authentication and complaint submission.



4.

4. Class Diagram

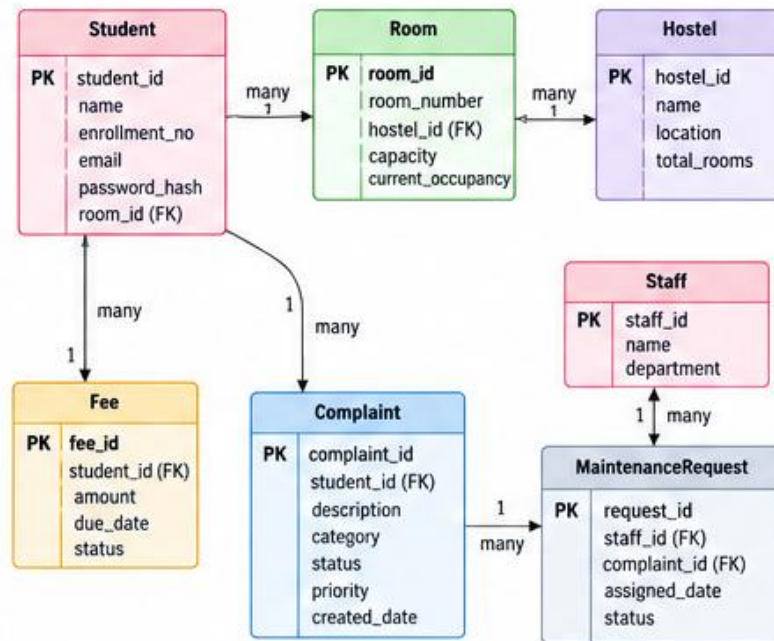
The Class Diagram represents the major classes of the Hostel Sync application and the relationships between them.



5. ER Diagram

The ER Diagram represents the relationships between the main data entities used by Hostel Sync.

5. ER Diagram



Design Decisions & Rationale

1. Choice of Java

Java was selected as the primary programming language for Hostel Sync because it provides strong support for object-oriented programming and allows the application to be organized into reusable and modular components. Its structured approach is suitable for implementing different entities, services, and application operations.

2. Use of Maven

Maven was used as the build and dependency management tool for the project. It provides a standardized project structure and simplifies dependency management, compilation, and execution of the application.

3. Layered Architecture

A layered architecture was chosen to separate different responsibilities within the application. Models represent system entities, services handle business logic, repositories manage

data access, and the application layer manages the overall workflow. This separation improves maintainability and makes individual components easier to modify.

4. Command-Line Interface

A command-line interface was selected to provide a simple and lightweight way for students and administrators to interact with Hostel Sync. It also allows the complete application to be executed directly through a terminal without requiring a graphical interface.

5. Complaint Classification

A complaint classification component was included to automatically categorize complaints based on their descriptions. Categories such as Plumbing, Electrical, Cleanliness, Internet, Furniture, and Other help organize maintenance issues and make them easier to manage.

6. Authentication and Password Hashing

Authentication was incorporated to control access to the application. Password hashing was used instead of storing passwords directly, providing an additional layer of protection for user credentials.

7. Modular Design

The project was divided into separate packages and components such as models, services, repositories, and AI-related components. This design reduces unnecessary dependency between parts of the application and makes the system easier to maintain and extend.

Implementation Details

Hostel Sync is implemented as a Java-based, Maven-managed application with a modular structure. The implementation separates the main application workflow, business logic, data models, repositories, authentication, and complaint analysis into different components.

1. Application Entry Point

The application starts through the main Java class and launches the Hostel Sync application workflow. The application provides a command-line menu through which users can access the available operations.

2. Model Components

The model layer represents the main entities used by the application. These include:

- User
- Student
- Staff
- Hostel
- Room
- Fee
- Complaint
- Maintenance Request
- Complaint Status
- Role

These classes organize the information required by different operations of the system.

3. Service Components

The service layer contains the business logic used to perform hostel management operations. It processes user requests and coordinates operations such as complaint management, student-related operations, fee handling, and dashboard functionality.

4. Repository Components

Repository components handle the storage and retrieval of application data. They provide a separate layer for accessing the information required by the services.

5. Authentication

The authentication functionality verifies user credentials and controls access to the application. The system supports different user

roles and uses password hashing for handling passwords.

6. Complaint Management

Students can submit hostel-related complaints through the application. Submitted complaints are stored with relevant information and can later be viewed through complaint history and complaint details.

Administrators can view submitted complaints and update their status, allowing the progress of maintenance issues to be tracked.

7. Complaint Classification

The complaint classification component analyzes the description entered by the student and assigns an appropriate category. The available categories include:

- Plumbing
- Electrical
- Cleanliness
- Internet
- Furniture
- Other

The system also includes priority prediction functionality for complaint analysis.

8. Administrator Dashboard

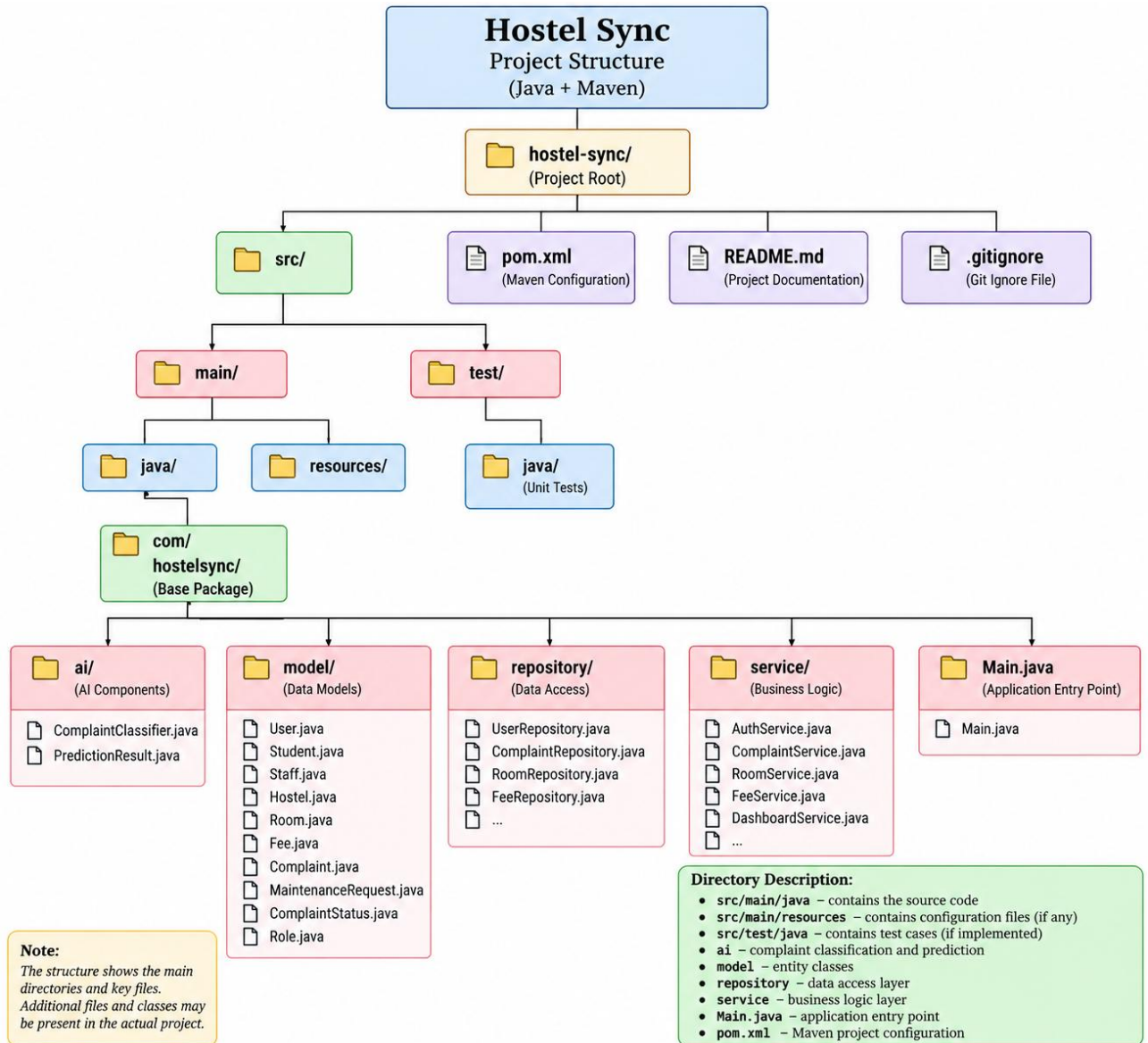
The administrator dashboard provides an overview of complaint information. It displays the total number of complaints along with category-wise and status-wise complaint information.

9. Command-Line Workflow

The complete application operates through the command line. Users authenticate themselves and are presented with options based on their role. The selected operation is processed by the relevant application and service components, and the result is displayed in the console.

10. Project Structure

The project follows a package-based structure that separates different responsibilities, including:



This structure keeps the implementation organized and supports easier maintenance and future expansion of the application.

Results

The Hostel Sync application was tested through its command-line interface to verify the main functionalities of the system. The following screenshots demonstrate the execution and results of the implemented features.

1. Application Start and Login

This screenshot shows the application starting successfully and displaying the login/authentication interface for the user.

```
Run Main x
C:\Users\ayush\.jdk\openjdk-26.0.2.1\bin\java.exe

=====
                HOSTELSYNC
        Smart Hostel Management
=====
1. Login
2. Demo
0. Exit
=====
Enter choice: 1
Enter email or phone: 7689765478

1. Student
2. Faculty
3. Keyless Entry
Select option: 1

=====
                HOSTELSYNC
        Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice:
```

2. Student Menu

This screenshot shows the options available to an authenticated student, including hostel information, room information, fee status, and complaint-related operations.

```
Run Main x
C:\Users\ayush\.jdk\openjdk-26.0.2.1\bin\java.exe

=====
                HOSTELSYNC
        Smart Hostel Management
=====
1. Login
2. Demo
0. Exit
=====
Enter choice: 1
Enter email or phone: 7689765478

1. Student
2. Faculty
3. Keyless Entry
Select option: 1

=====
                HOSTELSYNC
        Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice:
```

3. Complaint Submission

This screenshot demonstrates a student submitting a hostel-related complaint through the application.

```
=====
                HOSTELSYNC
            Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
9. Logout
=====
Enter choice: 3
Describe your complaint: he bathroom tap in Room 101 is leaking and needs immediate repair.

Complaint submitted successfully!
Category: Plumbing
Priority: HIGH
AI Confidence: 90.0%
```

4. Complaint Classification

This screenshot demonstrates the classification of a submitted complaint into an appropriate category.

```
=====
                HOSTELSYNC
            Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
9. Logout
=====
Enter choice: 6
Enter complaint for AI analysis: he bathroom tap in Room 101 is leaking and needs immediate repair.

--- AI ANALYSIS ---
Category: Plumbing
Priority: HIGH
Confidence: 90.0%
```

5. Complaint History / Status

This screenshot shows the student's complaint history and the current status of submitted complaints.

```
=====
HOSTELSYNC
Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice: 4

--- COMPLAINTS ---
#1 | Electrical | MEDIUM | PENDING
    The fan in my room is not working
#2 | Plumbing | HIGH | PENDING
    he bathroom tap in Room 101 is leaking and needs immediate repair.
```

6. Administrator Dashboard

This screenshot shows the administrator dashboard containing an overview of complaint information, including complaint statistics and category/status information.

```
=====
HOSTELSYNC
Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice: 7

===== HOSTEL DASHBOARD =====
Total Students      : 15000
Total Rooms         : 2000
Occupied Rooms      : 1800
Available Rooms     : 200
Occupancy Rate      : 90.0%
-----
Total Complaints    : 2
Pending Complaints  : 2
In Progress         : 0
Resolved Complaints : 0
-----
Unpaid Fees         : 1
=====
```

7. Complaint Status Update

This screenshot demonstrates an administrator updating the status of a complaint.

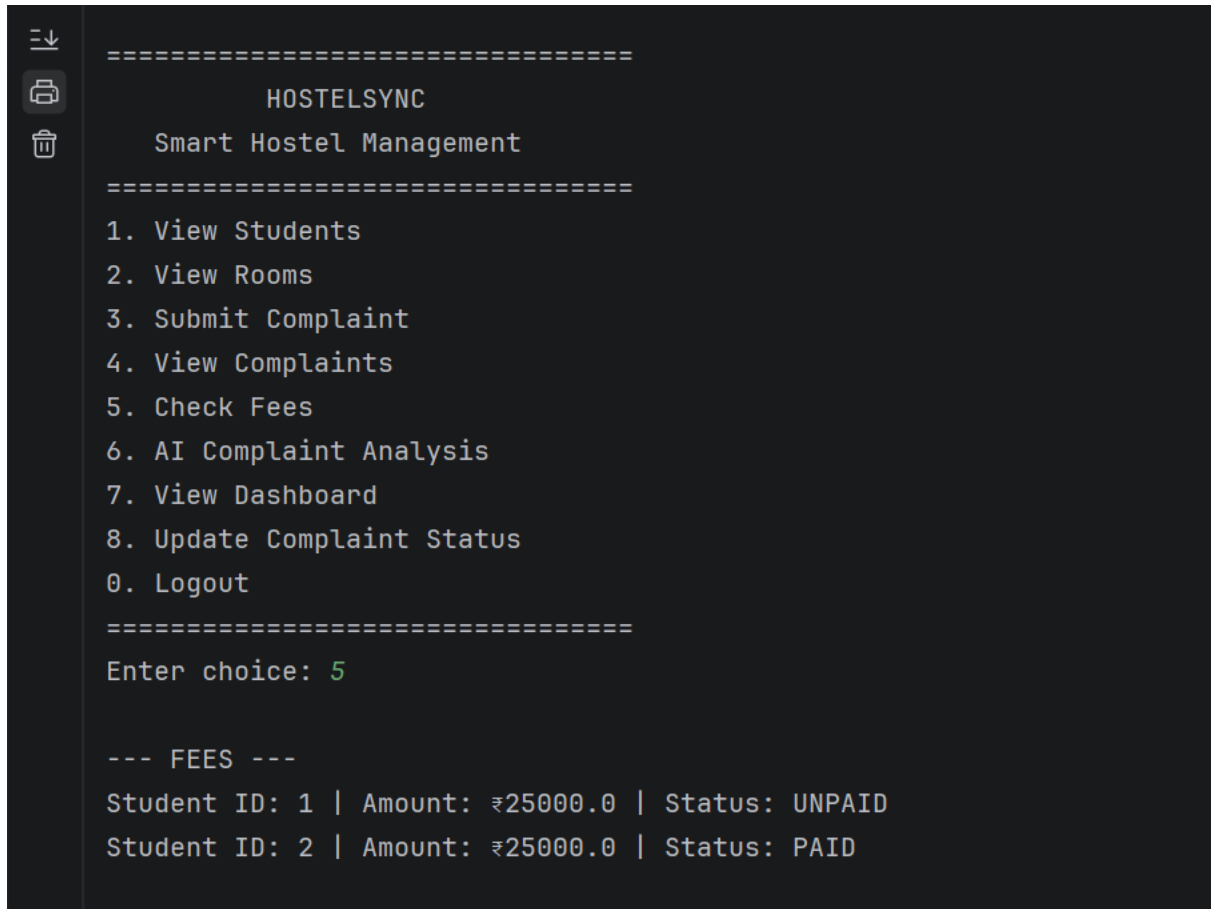
```
=====
                HOSTELSYNC
            Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice: 8
Enter complaint ID: 2

Current Status: PENDING
1. ASSIGNED
2. IN_PROGRESS
3. RESOLVED
4. CLOSED
Select new status: 3
Complaint #2 updated to RESOLVED
```

8. Fee Status

This screenshot shows the fee information for students, including the fee amount and

whether the fee has been paid or remains unpaid.



```
=====
                        HOSTELSYNC
                Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice: 5

--- FEES ---
Student ID: 1 | Amount: ₹25000.0 | Status: UNPAID
Student ID: 2 | Amount: ₹25000.0 | Status: PAID
```

8. Room Information

This screenshot shows the hostel room details, including room capacity, occupied spaces, and room availability.

```
=====
                        HOSTELSYNC
Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
9. Logout
=====
Enter choice: 2

--- ROOMS ---
Room 101 | Capacity: 2 | Occupied: 1 | Available: true
Room 102 | Capacity: 2 | Occupied: 2 | Available: false
```

9. Student Information

This screenshot shows the student records available in the system, including student names and their assigned room numbers.

```
=====
                        HOSTELSYNC
                Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice: 1

--- STUDENTS ---
1 | Ayush | Room 101
2 | Rahul | Room 102
```

10. Faculty Access

This screenshot shows the faculty login flow and the menu available after successful faculty access to the HostelSync application.

```
Run Main x
C:\Users\ayush\.jdk\openjdk-26.0.2.1\bin\java.exe "-javaagent:

=====
                        HOSTELSYNC
                Smart Hostel Management
=====
1. Login
2. Demo
0. Exit
=====
Enter choice: 1
Enter email or phone: 86868967899

1. Student
2. Faculty
3. Keyless Entry
Select option: 2

=====
                        HOSTELSYNC
                Smart Hostel Management
=====
1. View Students
2. View Rooms
3. Submit Complaint
4. View Complaints
5. Check Fees
6. AI Complaint Analysis
7. View Dashboard
8. Update Complaint Status
0. Logout
=====
Enter choice: |
```

Results

The implemented HostelSync system successfully provides the core hostel management and complaint management functionalities through a command-line interface. The application supports user login and role selection, allowing users to access the hostel management menu. The system provides student information, room details, fee status, complaint submission, complaint history, and complaint status management. The complaint submission feature also performs AI-based classification and priority analysis, helping organize hostel complaints into relevant categories. The dashboard provides an overview of complaint-related information for monitoring and management.

Testing Approach

The HostelSync application was tested through its command-line interface to verify that the implemented features work correctly with different inputs and user actions. Each major module was executed separately and the displayed output was checked against the expected result.

1. Login and Role Selection

The login functionality was tested by providing a phone number/email and selecting different available roles such as Student and Faculty. The system successfully accepted the input and

provided access to the corresponding application menu.

2. Student Information

The View Students option was tested to verify that student records and their assigned room numbers were displayed correctly.

3. Room Management

The View Rooms option was tested using the available room records. Room capacity, occupied spaces, and availability status were checked.

4. Fee Management

The Check Fees option was tested to verify that student fee amounts and payment statuses were displayed correctly.

5. Complaint Submission

A hostel complaint was entered through the complaint submission module. The system successfully stored the complaint and generated its category, priority, and AI confidence score.

6. Complaint Viewing

The View Complaints option was tested to verify that previously submitted complaints were displayed with their category, priority, description, and current status.

7. Dashboard and Status Management

The dashboard and complaint status functionality were tested to verify that complaint information could be monitored and that complaint statuses could be updated through the available options.

8. Input Validation

Different menu options and user inputs were tested to ensure that the application handles the available choices correctly and displays appropriate output for the selected operation.

Challenges Faced

The development of HostelSync involved several challenges while implementing and integrating the different modules of the application.

1. Managing Multiple Modules

Integrating student, room, fee, complaint, dashboard, and user-access functionalities required careful organization of the project components.

2. Handling User Input

Since the application operates through the command line, inputs had to be handled carefully so that

the correct operation was performed based on the user's selection.

3. Complaint Management

Designing the complaint workflow required managing complaint submission, categorization, priority, status, and viewing of previously submitted complaints.

4. AI-Based Complaint Analysis

Implementing the AI complaint analysis functionality required connecting the complaint input with classification and priority prediction while also displaying the confidence score.

5. Maintaining Project Structure

Keeping the model, repository, service, AI, exception, and application components organized was important for making the project easier to maintain and extend.

Learnings & Key Takeaways

Working on HostelSync helped in understanding the practical development of a modular Java application and the process of combining multiple functionalities into a single system.

1. Java Programming

The project provided practical experience with Java programming and object-oriented concepts while developing the different application components.

2. Modular Development

The project demonstrated how separating models, repositories, services, and application logic can make a project more organized and easier to maintain.

3. Application Workflow

Developing the command-line interface helped in understanding how user input, menu options, processing, and output can be connected to create a complete application workflow.

4. Complaint Management

The project provided experience in designing a complete complaint workflow, from submitting a complaint to categorizing it, assigning priority, viewing its status, and updating it.

5. AI Integration

Implementing AI-based complaint analysis provided practical understanding of how AI classification can be incorporated into an application to analyze user-submitted information.

6. Testing and Debugging

Running the application repeatedly helped identify issues, verify individual features, and improve the overall reliability of the system.

Future Enhancements

The current version of HostelSync provides the core hostel management and complaint management features. The system can be further improved with the following enhancements:

1. Database Integration

A permanent database can be integrated to store student, room, fee, and complaint records instead of relying on the current application data.

2. Web or Mobile Interface

A graphical web or mobile interface can be developed to make the system easier to access and use.

3. Online Fee Payment

Online payment functionality can be added so that students can pay their hostel fees directly through the system.

4. Notifications

Email or mobile notifications can be introduced to inform students about complaint updates, fee reminders, and other hostel-related information.

5. Advanced AI Analysis

The AI module can be expanded to provide more accurate complaint classification, priority prediction, and automated analysis of recurring hostel issues.

6. Enhanced Dashboard

The dashboard can be extended with charts and additional statistics to provide

better insights into rooms, fees, and complaint trends.

References

1.VITyarthi — Build Your Own Project Guidelines and Evaluation Requirements.

2.Oracle Java Documentation — Java programming language and standard library documentation.

3.Apache Maven Documentation — Project build and dependency management.

4.GitHub Documentation — Repository management and version control.

5.Documentation/resources used during the development of the AI-based complaint classification component.